

Particles and Weed into the Swampland: Testing Metaheuristics Ability for Vacua Search in String Theory

Anonymous Author

Abstract

String theory, born within the realm of high energy physics, has this major problem, which the Swampland conjectures had pronounced: it can predict well, but given the enormous amount of predicted vacua—most of them not satisfactory—, the need for a methodological search arose. Past researches used traditional gradient—based and sampling—based methods whose results were limited but not faulty. Recent researches focused on Metaheuristic algorithms—mainly genetic ones—as an option for exploring the landscape. As a result, new configurations were discovered, and the resulting points of interest increased in number.

This work aims to follow the metaheuristics way with the question: are there better algorithms for this problem? So, this approach involves two new —in this research line— algos: Adaptive Inertia Weight Particle Swarm Optimisation (AIW-PSO) and Invasive Weed Optimisation (IWO). Both population-based, both very different paradigms on a previously formulated effective potential within the II-B string theory. The performance here is assessed using an evenly-weighted Broda evaluation with 4 statistical-moments of the results distribution(V_{eff}, F).

Keywords and phrases: String Theory, Swampland, Bio-Inspired Algorithms, Nature-Inspired Algorithms, Metaheuristics, Optimisation, No Free Lunch Theorem, PSO, IWO, Broda

2020 Mathematics Subject Classification: 68N01

1 Introduction

There are several problems that cannot be easily solved by analytical methods or deterministic numerical algorithms due to their inherent complexity or their capricious, unpredictable behaviour—i.e., non-convex, rugged geometric landscapes. In the field of string theory, mathematical models predict an extensive landscape of possible universe configurations which, in most cases, do not correspond to our own—namely, those that are tachyonic or exhibit extreme potential energy values. Thus, many approaches in this field now involve artificial intelligence and machine learning frameworks [1], [2], [3], [4], [5]. **Conclusión:** La hoja editada demuestra que no necesitabas "reescribir desde cero", sino **despojar al texto de la capa de barniz comercial que la IA le había superpuesto**. El resultado es un borrador limpio, intelectualmente honesto y completamente blindado contra el escrutinio algorítmico del congreso.

Navigation of extensive string landscapes remains a challenge for identifying stable vacua within the swampland program. This benchmark evaluates optimization algorithms using three descriptors: statistical moments of the effective potential (V_{eff}), convergence of an V_{eff} -derived fitness function, and success ratios of stable vacua—both globally and within de Sitter (dS) points. To account for non-transitive performance criteria, an evenly weighted Borda rank [6] is applied—with a deliberate bias toward dS vacua for phenomenological requirements of the swampland program necessitated in this line of research.

2 Theoretical Framework and State of the Art

1

2.1 Genetic Algorithms in the String Landscape

A first approach can be found within the work of Abel and Rizos [7] where genetic algorithms were applied to explore string search spaces of 2^{51} configurations.

A posterior research by Damian and Loaiza-Brito [8], . What they found: 10 dS similar flux configurations. Later approaches used GAs for the exploration with different topological treats—for isotropic, semi-isotropic and anisotropic tori—[9]. So, at this point, GAs have been used as the main metaheuristic for venturing into landscape explorations.

2.2 Machine Learning-Enhanced Exploration

Cabo Bizet et al. [1] combined a GA with a supervised neural network to classify over 16 000 flux configurations in Type IIB compactifications on an isotropic torus. This surrogate classifier achieved 98.6% accuracy in identifying scalar potential critical points—this implied saving computational resources at the explorations. The points obtained showed concordance with the refined de Sitter conjecture: no stable dS vacua with critical points distribution consistently satisfying the bound $\min \nabla_i \nabla_j V \leq -c' V$ with $c' \sim 1$ [1].

2.3 Torsion Compactifications and the Simulated Annealing Baseline

The potential effective (Eq 1)—and most of the mathematical background—used in this work corresponds to the work of Damian and Loaiza-Brito defined in Type IIB compactifications on Kähler manifolds with torsion [2]:

$$V_{\text{eff}} = \frac{A_{H_3} s}{\tau^3} + \frac{A_{F_3}}{s \tau^3} + \frac{A_{F_5}}{\tau^4} + \frac{A_3 \mathcal{N}_3}{\tau^3} + \frac{A_{\widehat{D5}}}{s^{1/2} \tau^{5/2}} \quad (1)$$

where s and τ denote the axio-dilaton and volume moduli, and A_i represent the flux coefficients. Damian and Loaiza-Brito's approach employed Simulated Annealing combined with conjugate-gradient refinement. Approach that identified over 170 metastable de Sitter (dS) vacua, subject to the condition: $\mathcal{N}_{O3} > 4 \frac{\sqrt{A_{H_3} A_{F_3}}}{A_3} + 2 \mathcal{N}_{\text{flux}}$.

2.4 Population-Based GA Formulations in Torsion Potentials

The single-agent baseline was recently extended in [10], implementing a simple genetic algorithm (GA) over the effective scalar potential with torsion via the localized penalty paradigm [2]. By scaling a five-dimensional flux parameter space within uniform bounds of $[-100, 100]$, the

population dynamics of the GA mapped the landscape to isolate 11 979 critical points, including 987 stable dS and 2 309 stable AdS vacua[10]. An interesting behaviour was that of the convergence to negative values of the $A_3\mathcal{N}_3$ contribution coefficient without explicit constraints—matching orientifold physical expectations[2].

2.5 New proposals with IWO and AIW-PSO baed on NFLT

. For context, the NFLT [11] states that optimisation performance is not linked overall to the algorithm but in a complex relationship with the mathematical properties of the search space [2]—Wolpert used the term “problem class”. The potential function (Eq. 1) presents a very corrugated topography and high sensibility to very small numerical changes[10]. For such a function, it’s intuitive to think on population based rather than one point based algorithm.

Particle Swarm Optimization (PSO) uses a trajectory-based paradigm described by two velocity vectors based on cognitive and social momentum—likely as flocks, whose participants follow a leader but also changes direction on self-criteria [12]; though, in this approach, leaders tend to abdicate.

Invasive Weed Optimisation (IWO) mechanism emulated seed dispersion. When a seed finds “prolific lands” within the landscape, then the seeds tend to concentrate in such point.

3 Methods and Materials

For this work, it will be used the framework `SymPy`. This was done with two main purposes: a) to keep the new codebase as similar as possible to the original implementation, and b) to achieve the highest precision possible during the landscape exploration search.

It’s worth to mention that the implementation, originally written in Wolfram Mathematica, was re-implemented in this work as a native `Python` module. This language was selected due to its widespread adoption and its large, active AI/ML community, alongside the benefits of being a free, open-source environment. Moreover, its modular approach permitted a clean separation of the optimisation steps into different operational blocks.

3.1 Search Conditions and Constraint Formulation

The global multi-dimensional search was performed using fixed, preconfigured moduli pairs, restricting the optimiser’s degrees of freedom strictly to the five flux coefficients: A_{H_3} , A_{F_3} , A_{F_2} , $A_3\mathcal{N}_3$, and $A_{\widehat{D}_5}$ [10]. The optimisation was repeated 100 times per configuration across 88 distinct moduli (s, τ) pairs. Every moduli pair was obtained from a dataset given previous experiments from[2].

Four statistical moments were computed from the resulting points: **mean**, **standard deviation**, **skewness**, and **kurtosis**. The kurtosis was computed in 3 ways: i) whole distribution, ii) definite positive values, iii) definite negative values. For this benchmark, ii and iii are used as metrics. This decision was taken in regard of both AdS and dS distribution analysis—being this a more physics-oriented interpretation..

As in the work of Damian & Loaiza-Brito [2], penalisation is given by the sum of individual penalisations $\pi_i(x):F(x)=\sum \pi_i(x)$ where x is the individual to assess, $F(x)$ is the fitness function and $\pi_i(x)$ is the i -th penalisation. There are 5 penalisation functions.

Scalar potential extrema (so we know it's likely a minima), $\pi_1(x)=|\nabla V_{\text{eff}}(x)|^2$.

Penalisation for AdS vacua (avoiding definite negative values), $\pi_2(x)=|V_{\text{eff}}(x)-V_{\text{eff}}(x)|$.

To avoid tachyons, there are two approaches used—both on the Hessian matrix (also called mass matrix). By the trace $\pi_3(x)=|\text{tr}(H)|-\text{tr}(H)$ or by the discriminant $\pi_4(x)=|\Delta|-\Delta$, for $\Delta=\det(H)-\frac{1}{4}\text{tr}^2(H)$. Mathematically, both are related to the eigenvalues of the mass matrix—but less extensive than $\lambda_{1,2}, \lambda_{1,2}\Delta=\det(H)-\frac{1}{4}\text{tr}^2(H)\pi_4(x)=|\Delta|-\Delta$

And, to preserve the validity of the anti-brane uplifting mechanism, definite negative values of the $\widehat{D5}$ -brane contribution are penalised $\pi_5(x)=|A_{\widehat{D5}}|-A_{\widehat{D5}}$.

Based on the prior analysis, the optimisation benchmarks are guided by the following criteria, which define the desired architectural profile within the scalar potential landscape: **s1**) minimised near-to-0 V_{eff} mean values, **s2**) controlled negative-bound skewness directed towards the vacuum origin, **s3**) suppressed positive-tail kurtosis profiles to eliminate high-energy anomalies, **s4**) suppressed negative-tail kurtosis profiles to stabilise local vacuum fluctuations, **s5**) maximised overall stable configurations ratio, **s6**) maximised stable de Sitter (dS) vacua ratio, and **s7**) minimised fitness function convergence bounds. Criteria which will be evaluated by a Broda rank [6].

The ranking was made following an evenly weighted positional score allocation for scale invariance; in other words, all metaheuristics were sorted across independent rankings—where every configuration obtains a punctuation relative to its position. Punctuations follows a descendent numbering: $n-1$ points for the top-performing model and 0 points for the lowest—for $n \in \mathbb{N}$ being the total number of models ranked—, where the global consensus index B_i for each architecture is then computed via $B_i=\sum P_{ij}, i=\{0, \dots, n-1\}$.

3.2 Fitness function wall-clock test

The current pipeline is powered by `sympy`, `cupy/numpy`, and `mealpy./sympy` can handle and `lambdify` any symbolic approach; `mealpy` provides the largest collection of metaheuristics required for these benchmark researches, and `cupy` offers high-performance GPU acceleration support to evaluate the fitness function generated by `sympy`. Evaluation of this potential is inherently expensive in computational terms.

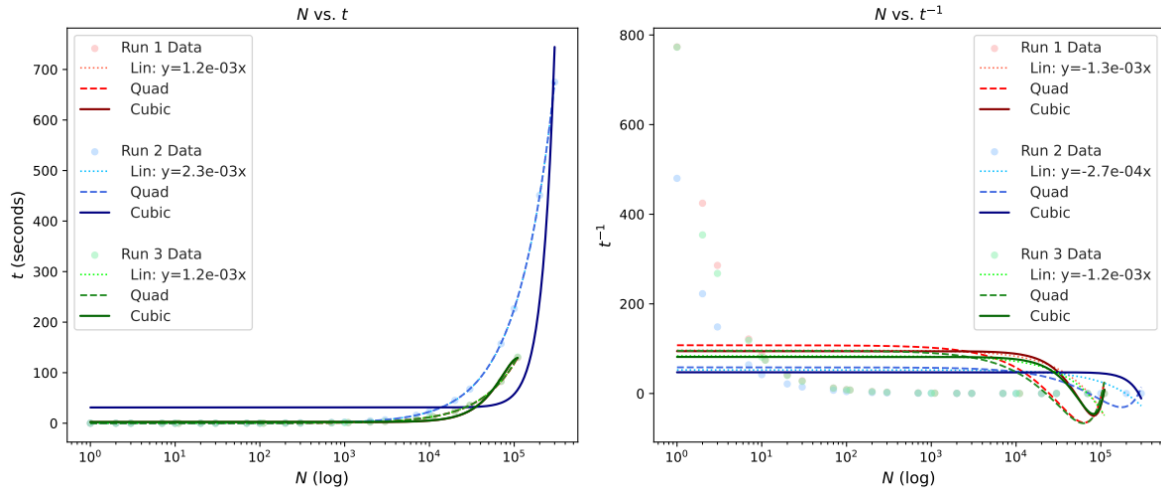
Since its complexity is assumed to be $\mathcal{O}(1)$ —for this function lying on plain arithmetical operations which shall not affect the computational complexity in other means but time—, a wall-clock test was performed to measure time cost. The results of the wall-clock test are shown in Table 1. What the table shows: number of executions N , time t , average seconds per execution $\bar{t}$, and executions per second t^{-1} .

After this, two additional tests were executed, and a linear regression was performed to compare N against both t and t^{-1} across all three trials. The regressions were plotted on a semi-logarithmic scale to accommodate the order-of-magnitude scaling of the iterations. Presumably, as shown in figure 1, the iteration throughput t^{-1} decreases with the increment of N before stabilising between 420 and 850 iterations per second, and the total execution time t scales linearly.

Table 1: Results of wall-clock test for the fitness function.

N	t	$\bar{t}$	t^{-1}	N	t	$\bar{t}$	t^{-1}
1	0.0012930	1.29E-03	773.42	300	0.3525780	1.18E-03	850.88
2	0.0023550	1.18E-03	849.25	700	0.8269840	1.18E-03	846.45
3	0.0034990	1.17E-03	857.33	1100	1.2978350	1.18E-03	847.57
7	0.0082160	1.17E-03	851.98	2000	2.3604520	1.18E-03	847.3
11	0.0133640	1.214935e-03	823.09	3000	3.5257810	1.18E-03	850.88
10	0.0114730	1.15E-03	871.58	7000	8.2698410	1.18E-03	846.45
20	0.0240690	1.20E-03	830.93	11000	12.9783510	1.18E-03	847.57
30	0.0352240	1.17E-03	851.69	20000	23.6045190	1.18E-03	847.3
70	0.0821420	1.17E-03	852.18	30000	35.2578080	1.18E-03	850.88
110	0.1285840	1.168944e-03	855.47	70000	82.6984450	1.18E-03	846.45
100	0.1172440	1.17E-03	852.93	110000	129.7835390	1.179850e-03	847.57
200	0.2352520	1.18E-03	850.15				

Figure 1: Polynomial regressions across three test executions confirm the reproducibility of the computational overhead.



3.3 IWO and PSO implementation

As no algorithm is proved to outperform in every possible problem [11]—here, it is assumed that this problem corresponds to a new class where it is needed to perform a search from scratch. Combinations selected to perform this benchmark are shown in table 2.

Table 2: Grid configurations for IWO and PSO, each configuration corresponds to a model.

Name	epoch	exponent	pop_size	seed_max	seed_min	σ_{end}	σ_{start}	Name	α	C_1	C_2	epoch	pop_size
------	-------	----------	----------	----------	----------	----------------	------------------	------	----------	-------	-------	-------	----------

IWO0	50	2	20	4	1	0.05	0.5	PSO0	0.4	0.02	0.02	50	20
IWO1	50	2	20	4	1	0.1	0.5	PSO1	0.4	0.02	0.04	50	20
IWO2	50	2	20	4	1	0.15	0.5	PSO2	0.4	0.02	0.08	50	20
IWO3	50	2	20	4	1	0.2	0.5	PSO3	0.4	0.02	0.2	50	20
IWO4	50	2	20	4	1	0.25	0.5	PSO4	0.4	0.02	0.4	50	20
IWO5	50	2	20	4	1	0.05	1	PSO5	0.4	0.04	0.02	50	20
IWO6	50	2	20	4	1	0.1	1	PSO6	0.4	0.04	0.04	50	20
IWO7	50	2	20	4	1	0.15	1	PSO7	0.4	0.04	0.08	50	20
IWO8	50	2	20	4	1	0.2	1	PSO8	0.4	0.04	0.2	50	20
IWO9	50	2	20	4	1	0.25	1	PSO9	0.4	0.04	0.4	50	20
IWO10	50	2	20	4	1	0.05	1.5	PSO10	0.4	0.08	0.02	50	20
IWO11	50	2	20	4	1	0.1	1.5	PSO11	0.4	0.08	0.04	50	20
IWO12	50	2	20	4	1	0.15	1.5	PSO12	0.4	0.08	0.08	50	20
IWO13	50	2	20	4	1	0.2	1.5	PSO13	0.4	0.08	0.2	50	20
IWO14	50	2	20	4	1	0.25	1.5	PSO14	0.4	0.08	0.4	50	20
IWO15	50	2	20	4	1	0.05	2	PSO15	0.4	0.2	0.02	50	20

Another assumption taken is that due to its high sensibility to decimal changes—in the order of $1e16$ or less, the Type IIB scalar potential necessitates a high-granularity hyperparameter search. Combinatorial analysis yields 1 000 configurations for AIW-PSO and 189 000 for Original-IWO families. This amount led the search to take only the first 16 configurations from each optimiser group (Table 2). Other imitations which led to the first ones are discussed later.

Nevertheless, the internal architecture of the `mealpy` framework is not natively designed to operate with `CuPy` data structures. In consequence of such limitation, penalisation functions ran over GPU in batch, later the sum returned to CPU where the algorithm operators were performed. Against any fear for bottleneck due to CPU↔GPU exchange (PCIe transfer latency), this performed far smarter than without it (from days to hours per optimiser for *cheaper* configurations).

For the AIW-PSO variant —let’s call it S —, the temporal cost per epoch remains strictly deterministic and invariant throughout the execution. Conversely, Original-IWO —let’s call it W — exhibits significant temporal volatility, as its dynamic, fitness-driven seed dispersion fluctuates according to the topolgraphy of the landscape.

3.4 Computational Complexity and Architectural Latency Analysis

The global computational cost must be modelled by coupling the metaheuristic operators with the physical hardware constraints. To do so, let $D=5$ represent the dimensionality of the scalar potential to optimise, $iter$ denote the current epoch up to $iter_{\max}=50$; and P , the population size.

For the S family, based on Qin et al. [12], the population size remains strictly invariant: $P_{\text{active}}(iter)=P, \forall iter \in [0, iter_{\max}]$.

For each run, the amount of GPU routings is directly the number of individuals : $N_{\text{evals}}^S = P$.

W family introduces a dynamic, fitness-driven population growth. Following the ecological mechanics established by Mehrabian & Lucas [13], the seed production Φ_i of the i -th individual plant is a linear projection of its normalised scalar fitness $f_i \in [f_{\min}, f_{\max}]$

$$: \Phi_i(iter) = l \Phi_{\min} + \frac{f_i - f_{\min}}{f_{\max} - f_{\min}} (\Phi_{\max} - \Phi_{\min}) J.$$

prior to competitive elimination is highly time-variant and demographically unstable: $P_{\text{interim}}(\text{iter}) = P(\text{iter}) + \sum \Phi_i(\text{iter})$. In other words, is as stochastic as the function topography.

Necessitated GPU calls here behaves in a more stochastic way, thus the total number of hardware evaluation operations W performs per epoch is given by $N_{\text{evals}}^W(\text{iter}) = P_{\text{interim}}(\text{iter}) \in [P_{\text{min}}, P_{\text{max}} + P_{\text{max}} \Phi_{\text{max}}]$.

Let τ_{exec} be the computational clock time for the symbolic GPU evaluation of a single position vector, and let τ_{bus} represent the bus latency and synchronisation overhead for the GPU $\leftrightarrow$ CPU exchange.. The total wall-clock time spent on evaluations per epoch ($\mathcal{T}_{\text{eval}}$) is expressed in Eqns. 2 and 3.

$$\mathcal{T}_{\text{eval}}^S = P \cdot (\tau_{\text{exec}} + \tau_{\text{bus}}) \quad (2)$$

$$\mathcal{T}_{\text{eval}}^W(\text{iter}) = P_{\text{interim}}(\text{iter}) \cdot (\tau_{\text{exec}} + \tau_{\text{bus}}) \quad (3)$$

Translating Eqns. 2 and 3 into asymptotic Big-O ($\mathcal{O}$) notation, the global complexity profiles $\mathcal{C}$ per epoch takes the forms shown in Eqns. 4 and 5.

$$\mathcal{C}_S = \mathcal{O}(P \cdot D + P \cdot (\tau_{\text{exec}} + \tau_{\text{bus}})) \quad (4)$$

$$\mathcal{C}_W = \mathcal{O}(P_{\text{interim}}(\text{iter}) \cdot D + P_{\text{interim}}(\text{iter}) \log P_{\text{interim}}(\text{iter}) + P_{\text{interim}}(\text{iter}) \cdot (\tau_{\text{exec}} + \tau_{\text{bus}})) \quad (5)$$

Within W , $P_{\text{interim}}(\text{iter}) \log P_{\text{interim}}(\text{iter})$ is the competitive sorting algorithm (Timsort) utilised to prune excess weeds. $P_{\text{interim}}(\text{iter}) \cdot \tau_{\text{bus}}$ acts as a multiplicative overhead.

3.5 About Computational Resources

The **processing node** utilised was an **Alienware** desktop with an **Intel Core i7-13700F processor** (13th Generation) , 16 GiB of **RAM**, 1 TiB **SK Hynix PC811 NVMe solid-state drive** as primary **storage**; **NVIDIA GeForce RTX 4060 graphics processing unit (GPU)**.

The operative system was a 64-bit **Void Linux** distribution running a monolithic **Linux kernel** (version 6.12.82) complemented with the proprietary **NVIDIA driver** (version 580.142).

4 Experiments and Results

After running all 32 optimisers, their results were saved in independent datasets which were later analysed. For the analysis, the resulting datasets were cleaned from outliers. The distribution of the effective potential was plotted (Figure 2), where it can be seen that the majority of the points were grouped near zero. Based on this, the 4 moments (5, accounting kurtosis separated analysis) were computed over each dataset.

The ranking was performed in three ways: b1) the best optimiser per descriptor (Table 3); b2) the best per descriptor pair (Table 4); b3) the evenly-weighted Broda ranking. Homogeneity in b1 and b2 clues the presence of Condorcet cyclic paradoxes. Thus, b3 is, indeed, needed.

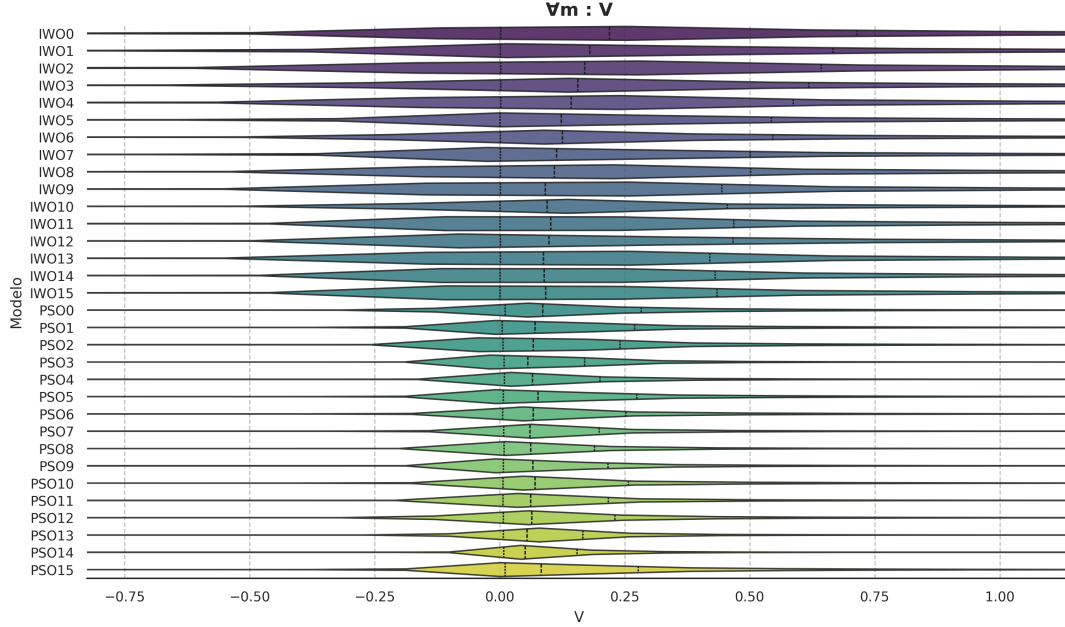


Figure 2: Distribution of the effective potential values.

Table 3: Individual Winners

Criterion	Winner	Value
s1	PSO14	0.0901948358
s2	IWO1	0.7400678318
s3	IWO0	4.8921034412
s4	PSO13	-2.1034455829
s5	IWO1	0.3981818182
s6	PSO12	0.2845610236
s7	PSO14	0.0276629053

Table 4: Pair-Combinatorial Tournament Matrix

Criterion	s1	s2	s3	s4	s5	s6	s7
s1	—	PSO4	PSO14	PSO13	IWO9	PSO7	PSO14
s2		—	IWO1	IWO7	IWO1	PSO0	IWO10
s3			—	PSO14	IWO1	PSO0	PSO14
s4				—	IWO13	PSO7	PSO13
s5					—	PSO1	PSO14
s6						—	PSO7
s7							—

From Marquez-Salazar work [10] some criterion (**s5**, **s6**) were taken. Comparing them with current work's results (Table 5), specially with **s6**, there is a concordance with swampland conjectures. Though the global ratio of stable vacua increased in this approach, de Sitter stable ratio didn't.

Table 5: **s5** and **s6** criterion comparison between a previous work and current work.

Criterion	s5	s6
Previous work's best	0.2751481759	0.2994538834
Current work's best	0.3981818182	0.2845610236

After performing b3, **PSO14** resulted as the global consensus winner.

5 Discussion

In regard of findings. Though results favoured AdS vacua, this work does not declare any superiority between compared optimisers, outside the 32 hyperparameter combinations plus previous GA explorations, there are more possibilities which shall be tested. This work intended also, a more formal computational comparison for this line of research which previous works lacked —for most of them being pioneers not computing-focused.

In regard of the selection of the 16 first configurations for each optimiser. Unfortunately, higher numbers on population and epochs tended to increase time, with experiments that lasted weeks and could not be finished. Characterising

In regard of infrastructure. Most of the time-extensive experiments could not be finished due to electrical cuts in the laboratory building. Though the process ended with no restoration point, half-filled datasets were saved. The importance of that is in regard of future dataset analysis where each point counts.

6 Conclusions

Notorious is that the question guided by the NFLT remains open. Broda rankings results (b1 and b2) demonstrated that each criteria was handled different between optimisers. Further research is needed to compare a higher variety of configurations, even of optimisers.

It's needed to highlight that for b3, s6 biased the Broda rank to benefit those optimisers whose points where, relatively, in majority dS. Which is a requirement needed to explore the landscape. Even though stable AdS vacua are interesting, in this work they are better avoided. This must be considered for researches where vacua types are important and a less biased exploration is needed.

Further experiments comparing metaheuristic optimisers are needed to confirm Broda's—and current criteria—powerfulness in regards of this research line as well as finding better metaheuristic optimisers and optimisers configurations.

Acknowledgements

The first author acknowledges the support from SECIHTI for the master's scholarship linked to the CVU 1295151.

References

- [1] N. C. Bizet, C. Damian, O. Loaiza-Brito, D. K. Mayorga Peña, and J. A. Montañez-Barrera, "Testing swampland conjectures with machine learning," *Eur. Phys. J. C*, vol. 80, no. 8, Aug. 2020, doi: 10.1140/epjc/s10052-020-8332-9.
- [2] C. Damian and O. Loaiza-Brito, "Metastable vacua from torsion and machine learning," *Eur. Phys. J. C*, vol. 82, no. 12, Dec. 2022, doi: 10.1140/epjc/s10052-022-11118-x.
- [3] X. Gao and H. Zou, "Applying machine learning to the Calabi-Yau orientifolds with string vacua," *Phys. Rev. D*, vol. 105, no. 4, Feb. 2022, doi: 10.1103/physrevd.105.046017.
- [4] G. B. De Luca, "Machine Learning gravity compactifications on negatively curved manifolds," *J. High Energy Phys.*, vol. 2025, no. 9, Sep. 2025, doi: 10.1007/jhep09(2025)090.
- [5] Y.-H. He, V. Jejjala, and T. S. R. Silva, "Metaheuristic generation of brane tilings," *Phys. Lett. B*, vol. 862, p. 139365, Mar. 2025, doi: 10.1016/j.physletb.2025.139365.
- [6] D. G. Saari, "Explaining All Three-Alternative Voting Outcomes," *J. Econ. Theory*, vol. 87, no. 2, pp. 313–355, Aug. 1999, doi: 10.1006/jeth.1999.2541.
- [7] S. Abel and J. Rizo, "Genetic algorithms and the search for viable string vacua," *J. High Energy Phys.*, vol. 2014, no. 8, Aug. 2014, doi: 10.1007/jhep08(2014)010.

- [8] C. Damian, L. R. Díaz-Barrón, O. Loaiza-Brito, and M. Sabido, "Slow-roll inflation in non-geometric flux compactification," *J. High Energy Phys.*, vol. 2013, no. 6, Jun. 2013, doi: 10.1007/jhep06(2013)109.
- [9] C. Damian and O. Loaiza-Brito, "More stable de Sitter vacua from S -dual nongeometric fluxes," *Phys. Rev. D*, vol. 88, no. 4, Aug. 2013, doi: 10.1103/physrevd.88.046008.
- [10] B. Marquez-Salazar, C. Damian, and A. Diaz-Pacheco, "Genetic Algorithm Search for Stable Vacua in Type IIB String Theory: A Computational Approach," 2026.
- [11] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 67–82, Apr. 1997, doi: 10.1109/4235.585893.
- [12] Z. Qin, F. Yu, Z. Shi, and Y. Wang, "Adaptive Inertia Weight Particle Swarm Optimization," in *Artificial Intelligence and Soft Computing – ICAISC 2006*, Springer Berlin Heidelberg, 2006, pp. 450–459. doi: 10.1007/11785231_48.
- [13] A. R. Mehrabian and C. Lucas, "A novel numerical optimization algorithm inspired from weed colonization," *Ecol. Inform.*, vol. 1, no. 4, pp. 355–366, Dec. 2006, doi: 10.1016/j.ecoinf.2006.07.003.